replace Claim 7.6 by $E'_q \vdash q.w'_q.w_q$. For similar reasons, Claim 7.7 must be replaced by the stronger $WI_{[E'_q.q]}(w'_q.w_q) \cap Sleep' = \emptyset$, which is actually also proven in the text of the proof of Claim 7.7. Continuing along these lines, the text of Claim 7.8 should be changed into:

> *Let z' be any sequence such that $E'_q.q.w'_q.w_q.z'$ is maximal. (Such a z' can always be found, since $E'_q.q.w'_q.w_q$ is an execution sequence.) Then, the algorithm explores some sequence $E'_q.q.z$ in $[E'_q.q.w'_q.w_q.z']_{\simeq}$.*

After the proof of Claim 7.8, the algorithm will handle the new kind of race (lines 8–14) between `lock(l)` operations, with $v = w.p$. At the end, we end up with the same Claim 7.9 as in the proof, i.e., $w'_q.w_R \sqsubseteq_{[E'_q]} v$, where the last process in w_R is actually $\widehat{e_q}$. Here, it is important to observe that e_q, being of form `lock(l)`, is actually independent with all events in w except possibly the ones already in $w'_q.w_q$. Thereafter, the proof goes on just as in the non-blocking case.

8.2 Allowing Arbitrary Blocking in the Execution Model

Let us next consider how to extend algorithms Source-DPOR and Optimal-DPOR to a general execution model, where disabling between processes can happen via any operations, in a computational model which need only satisfy the requirements of Definition 3.2. If disabling between processes happens only via clearly identified synchronization mechanisms, such as locks, Algorithms 3 and 4 with obvious modifications are sufficient. However, if disabling can occur in less obvious ways, the modification is less straightforward. The reason is that in the general case, races may be more difficult to detect. We illustrate this by a small example.

Initially: `x = y = 0`

p :	q :	r :	s :
`x +:= 1;`	`x +:= 1;`	`x +:= 3;`	`await (x = 3)` `then y := 1;`

Fig. 9. Example motivating line 11 of Algorithm 6.

Consider the program in Fig. 9. All four statements are atomic. In this example, we define the following happens-before relation on the events in an execution sequence: let the statement of s be related with any other statement, and let the statements of processes p, q, and r all be unrelated if the statement of s does not occur somewhere between them. The statements of processes p, q, and r are all related if the statement of s occurs between them. Such a happens-before relation is somewhat non-standard; another natural choice of happens-before relation would be to let it relate all pairs of statements, since they all touch the variable `x`. However, our choice is also consistent with the requirements on a happens-before relation in Definition 3.2, since the order in which the atomic statements of p, q, and r are performed does not matter when they occur consecutively. With this happens-before assignment, the program has only two Mazurkiewicz traces: one in which only p, q, and r occur (whereafter s is blocked), and one consisting of $r.s.p.q$ and $r.s.q.p$.

To illustrate the difficulty of detecting races, suppose that we first explore the sequence $p.q.r$, after which s is blocked. In some way, we must now define a "race". There is a race between p and s, but such a race is visible only in explorations that start with $r.p$. More generally, this example shows that in the performed execution sequence (in this case $p.q.r$), one may have to extract a subsequence (in this case r) in order to make the race visible. The subsequence r should clearly be consistent with $p.q.r$ in the sense that $r \sqsubseteq_{[\langle\rangle]} p.q.r$.